Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

SUMIRAN VUPPULURI	**PES2UG24CS536**
SURAJ ACHARYA	**PES2UG24CS539**

under the guidance of

Dr Farida Begum

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

SUMIRAN VUPPULURI	**PES2UG24CS536**
SURAJ ACHARYA	**PES2UG24CS539**

In partial fulfilment for the completion of fourth semester OS Project (UE24CS242B) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4^{th} semester academic requirements in respect of project work.

ACKNOWLEDGEMENT

I would like to express my gratitude to Dr. Farida Begum, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	1
2.	PROBLEM DEFINITION	2
3.	METHODOLOGY	3
4.	CODE	4
5.	RESULT ANALYSIS	6
6.	CONCLUSION	19
7.	GITHUB LINK	20
8.	REFERENCES	20

INTRODUCTION:

Containers have become the backbone of modern software deployment, powering everything from microservices to cloud-native applications. Yet most developers interact with them only through high-level tools like Docker or Podman, with little understanding of the Linux kernel primitives that make containerization possible in the first place. At their core, containers are nothing more than carefully isolated processes separated using namespaces, constrained by cgroups, and managed by a runtime that coordinates their entire lifecycle. This project strips away those abstractions and builds a lightweight container runtime from the ground up in C, exposing exactly how the OS achieves process isolation, resource enforcement, and concurrent process management.

The runtime is built around a long-running parent supervisor that launches and tracks multiple containers simultaneously, each isolated using PID, UTS, and mount namespaces with an Alpine Linux root filesystem. A bounded-buffer logging pipeline captures container output safely across concurrent producers and consumers, while a CLI allows real-time interaction with the supervisor over a dedicated IPC channel. On the kernel side, a custom Linux Kernel Module monitors per-container memory usage and enforces soft and hard limits via ioctl, directly integrating kernel-space enforcement with user-space metadata. Together, these components form a cohesive system that not only demonstrates container mechanics but also serves as an experimental platform for observing Linux scheduling behavior under different workload configurations.

PROBLEM DEFINITION:

Modern cloud and containerized environments rely heavily on container runtimes to isolate and manage processes efficiently, yet understanding how these systems work at the OS level remains a significant gap for most developers. Existing tools like Docker abstract away the underlying mechanics namespaces, process supervision, IPC, memory enforcement, and scheduling making it difficult to reason about what actually happens at the kernel level. This project addresses that gap by building a lightweight Linux container runtime from scratch in C, forcing a direct engagement with the core OS primitives that production runtimes are built on.

The challenge lies not just in achieving isolation, but in safely managing multiple concurrent containers, coordinating logging without data races, enforcing memory limits from kernel space, and observing real scheduling behavior all of which require a deep, hands-on understanding of how a modern Linux OS actually works. Unlike simply running containers, building one demands answers to questions that are easy to ignore otherwise: How does a parent process safely reap children without creating zombies? How do you pass data between processes without corruption under concurrency? When and why should memory enforcement happen in kernel space rather than user space? This OS Jackfruit problem is an attempt to answer all of these questions through implementation, using a working multi-container runtime as both the deliverable and the proof.

METHODOLOGY:

The implementation follows a bottom-up, incremental approach across six tasks, all built on top of the boilerplate in C. The project is structured around two core components engine.c for user-space and monitor.c for kernel-space with shared ioctl definitions in monitor_ioctl.h bridging them.

The first step was setting up the environment on an Ubuntu VM with Secure Boot disabled, installing kernel headers, and running the provided environment-check.sh to verify the build toolchain. The Alpine Linux mini root filesystem was prepared separately and used as the container root for all isolation experiments.

The user-space runtime in engine.c was built around a long-running supervisor process. Each container is spawned as a child using clone() with CLONE_NEWPID, CLONE_NEWUTS, and CLONE_NEWNS flags to achieve namespace isolation, followed by chroot into the Alpine rootfs. The supervisor maintains a metadata table for each container tracking host PID, state, memory limits, log path, and exit status protected by a mutex for safe concurrent access. SIGCHLD handling was implemented to correctly reap exited children and prevent zombie processes.

Container stdout and stderr are captured via pipes into a bounded-buffer logging system, where producer threads insert log data and a consumer thread drains it to per-container log files. Synchronization is handled with mutexes and condition variables. A separate UNIX domain socket serves as the control channel between the CLI client and the supervisor, allowing commands like start, stop, ps, and logs to be issued in real time satisfying the requirement for a second distinct IPC mechanism.

The kernel module in monitor.c was developed and loaded separately. It exposes /dev/container_monitor, through which the supervisor registers container host PIDs via ioctl. The module maintains a kernel linked list of monitored processes, protected by a spinlock, and periodically checks RSS usage. Crossing the soft limit triggers a dmesg warning; crossing the hard limit sends a kill signal to the process and the supervisor updates the container's metadata to reflect the kill reason.

Scheduling experiments were conducted using the provided cpu_hog.c and io_pulse.c workloads, run as containers simultaneously under different nice values and CPU affinities. Completion times and CPU share were measured and compared to analyze CFS scheduler behavior across competing workload types.

CODE EXPLANATION:

The project has two integrated parts:

1. User-Space Runtime + Supervisor (engine.c) Launches and manages multiple isolated containers, maintains metadata for each container, accepts CLI commands, captures container output through a bounded-buffer logging system, and handles container lifecycle signals correctly.

The engine.c file implements a miniature container runtime in a single C file. The main function dispatches to either the supervisor mode or one of the CLI commands (start, run, ps, logs, stop). The supervisor is the long-running daemon it creates a Unix domain socket at /tmp/mini_runtime.sock, installs signal handlers for SIGCHLD and SIGINT/SIGTERM, spins up a logging thread, and then loops on select waiting for client connections. When a CMD_START request arrives, it calls launch_container, which allocates a stack, creates a pipe for log capture, and uses clone with CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS to spawn the container process in isolated namespaces.

The child (child_fn) sets its hostname, mounts /proc twice (before and after chroot), chroots into the rootfs, redirects its stdout/stderr into the pipe's write end, applies any nice value, and then execs the command. Back in the supervisor, a detached log_reader_thread reads from the pipe's read end and pushes chunks into a bounded buffer (a circular array of 16 slots with a mutex and two condition variables).

The dedicated logging_thread pops from that buffer and appends each chunk to the container's log file. The SIGCHLD handler reaps exited children with waitpid in a non-blocking loop and updates each container's state in the linked list of container_record_t structs, distinguishing between a manual stop and a hard-limit kill based on whether stop_requested was set.

On shutdown, the supervisor signals all running containers, drains the log buffer, joins the logging thread, frees the entire container linked list, closes the socket, unlinks the socket path, and optionally closes the kernel monitor's file descriptor leaving no zombies, no open handles, and no heap leaks.

2. Kernel-Space Monitor (monitor.c) Implements a Linux Kernel Module (LKM) that tracks container processes, enforces soft and hard memory limits, and integrates with the user-space runtime through ioctl.

monitor.c is a Linux kernel module that acts as a memory watchdog for the container runtime. It registers a character device at /dev/container_monitor which the supervisor (engine.c) communicates with via ioctl. The core data structure is a linked list of monitored_entry nodes, each tracking a container's PID, its ID string, soft and hard memory limits, and a flag for whether the soft warning has already been emitted. A spinlock protects this list across the two code paths that touch the ioctl handler and the timer callback.

When the supervisor launches a container, it calls ioctl(MONITOR_REGISTER), which allocates a new node with kmalloc, validates that soft limit doesn't exceed hard limit, and appends it to the tail of the list under the spinlock. Every second, the timer callback fires, iterates the list using list_for_each_entry_safe (the safe variant is necessary because entries may be deleted mid-iteration), and for each entry calls get_rss_bytes which walks the kernel's mm_struct to read the process's resident set size in pages and converts it to bytes. If the process has already exited, get_rss_bytes returns -1 and the entry is removed and freed. If RSS exceeds the hard limit, send_sig(SIGKILL) is sent to the task and the entry is freed. If RSS exceeds the soft limit and the warning hasn't been emitted yet, it logs a KERN_WARNING and sets soft_warned = 1 so it only fires once. On rmmod, monitor_exit first calls timer_shutdown_sync to ensure the timer callback has fully stopped before touching the list, then walks the entire list freeing every remaining node with kfree, and finally tears down the cdev, device, class, and character device region in reverse order of how they were created — leaving no leaked kernel heap memory or stale device nodes.

RESULT ANALYSIS

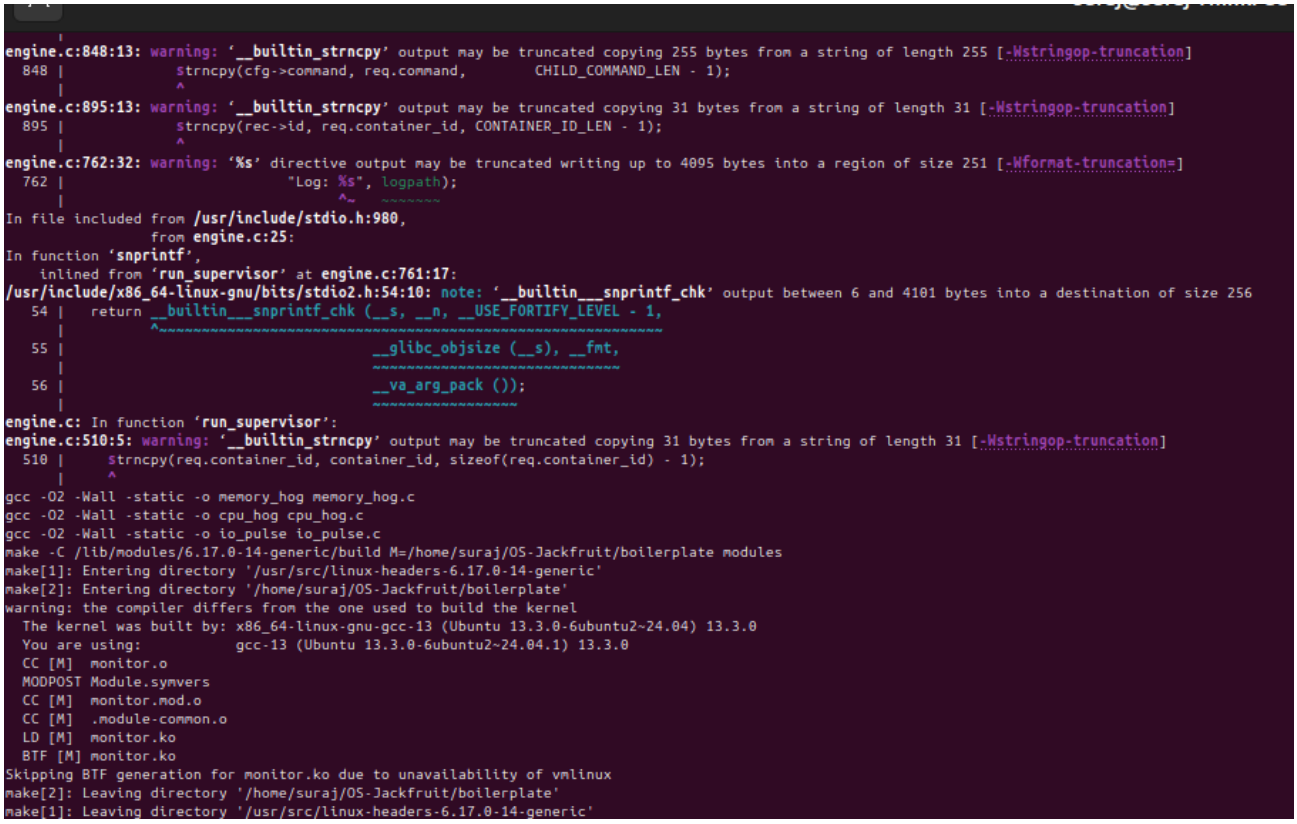
Task 1: Multi-Container Runtime with Parent Supervisor

The goal of Task 1 is to implement a parent supervisor process capable of managing multiple containers simultaneously, instead of launching a single process and exiting. The supervisor remains active for the entire lifecycle of all containers, enabling concurrent execution and centralized control.Each container is created using the clone() system call with namespace flags CLONE_NEWPID, CLONE_NEWUTS, and CLONE_NEWNS, ensuring isolation of process IDs, hostnames, and mount points. This guarantees that processes inside one container cannot see or interfere with those in another.

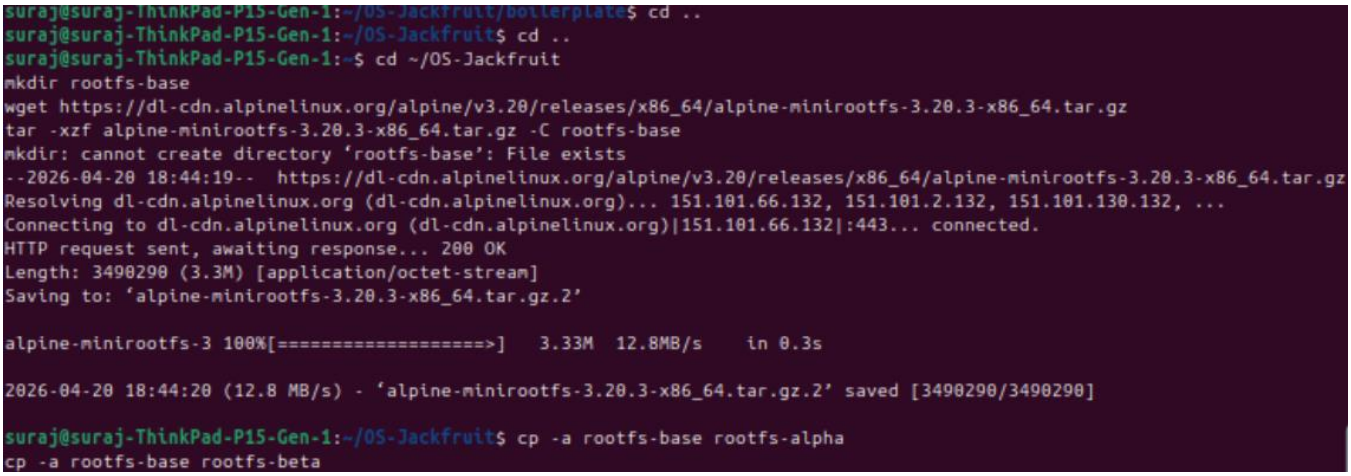
For filesystem isolation, rootfs-base/ is used as a template, and each container is assigned its own writable copy (e.g., rootfs-alpha/, rootfs-beta/). The chroot system call is used to restrict each container to its respective root filesystem, making it appear as the root directory (/). Additionally, /proc is mounted inside each container using mount("proc", "/proc", "proc", 0, NULL) to enable process-related utilities such as ps to function correctly.The supervisor maintains a metadata structure in user space for each container, tracking details such as container ID, host PID, start time, current state (e.g., running, stopped, or killed), configured soft and hard memory limits, log file path, and exit status. This metadata is protected using mutexes to ensure safe concurrent access.

The supervisor also handles SIGCHLD signals correctly by reaping all terminated child processes using waitpid(), thereby preventing zombie processes and ensuring proper lifecycle management.

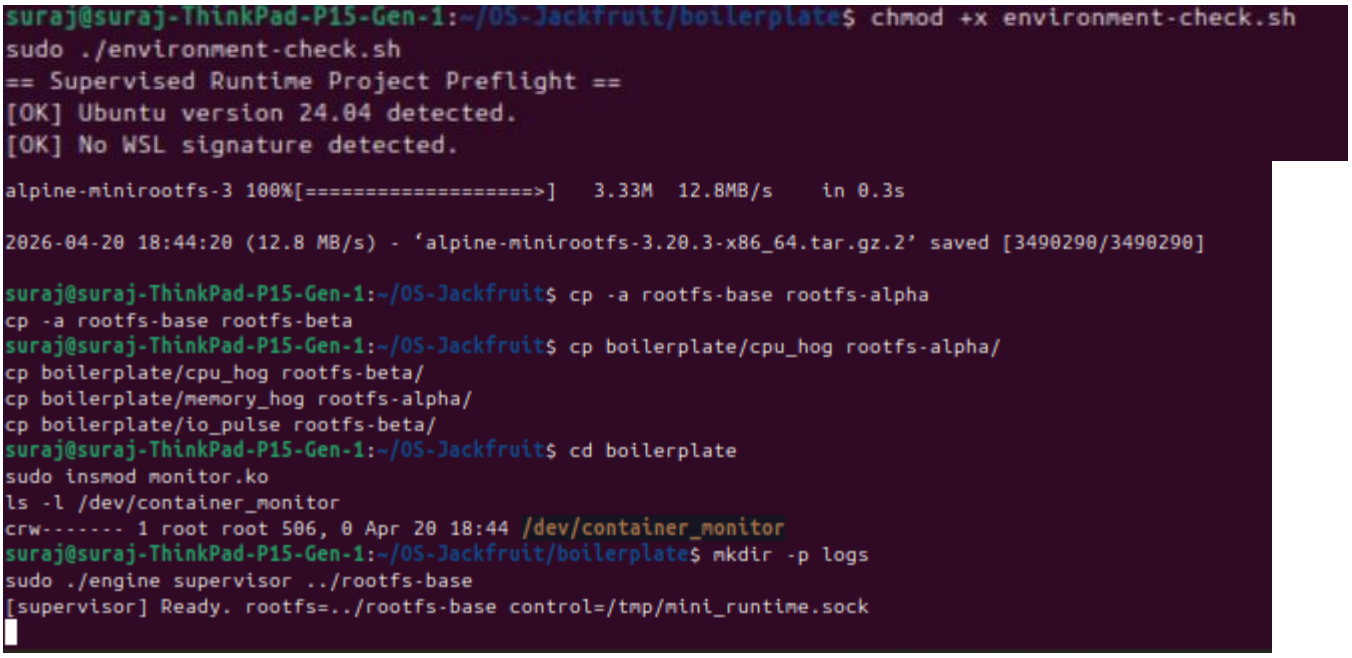
```
suraj@suraj-ThinkPad-P15-Gen-1:~/OS-Jackfruit/boilerplate$ sudo make clean
if [ -d "/lib/modules/6.17.0-14-generic/build" ]; then make -C /lib/modules/6.17.0-14-generic/build M=/home/suraj/OS-Jackfruit/boilerplate clean; fi
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-14-generic'
make[2]: Entering directory '/home/suraj/OS-Jackfruit/boilerplate'
  CLEAN   Module.symvers
make[2]: Leaving directory '/home/suraj/OS-Jackfruit/boilerplate'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-14-generic'
rm -f engine memory_hog cpu_hog io_pulse *.o *.mod *.mod.c *.symvers *.order
rm -f *.log
rm -rf logs
rm -f /tmp/mini_runtime.sock
suraj@suraj-ThinkPad-P15-Gen-1:~/OS-Jackfruit/boilerplate$ make
gcc -O2 -Wall -Wextra -o engine engine.c -lpthread
engine.c: In function ‘logging_thread’:
engine.c:392:9: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  392 |         write(fd, item.data, item.length);
      |         ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c: In function ‘child_fn’:
engine.c:474:9: warning: ignoring return value of ‘nice’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  474 |         nice(cfg->nice_value);
      |         ^~~~~~~~~~~~~~~~~~~~~
engine.c: In function ‘run_supervisor’:
engine.c:758:17: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  758 |                 write(client_fd, &resp, sizeof(resp));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:763:17: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  763 |                 write(client_fd, &resp, sizeof(resp));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:769:21: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  769 |                     write(client_fd, buf, (size_t)nr);
      |                     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:814:17: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  814 |                 write(client_fd, &resp, sizeof(resp));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:826:17: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  826 |                 write(client_fd, &resp, sizeof(resp));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:841:17: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  841 |                 write(client_fd, &resp, sizeof(resp));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:875:17: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  875 |                 write(client_fd, &resp, sizeof(resp));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:890:17: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  890 |                 write(client_fd, &resp, sizeof(resp));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:955:17: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  955 |                 write(client_fd, &resp, sizeof(resp));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:970:17: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  970 |                 write(client_fd, &resp, sizeof(resp));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c:981:9: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  981 |         write(client_fd, &resp, sizeof(resp));
      |         ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
engine.c: In function ‘pipe_reader_thread’:
engine.c:410:5: warning: ‘__builtin_strncpy’ output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
  410 |     strncpy(item.container_id, pra->id, CONTAINER_ID_LEN - 1);
      |     ^
engine.c: In function ‘run_supervisor’:
engine.c:846:13: warning: ‘__builtin_strncpy’ output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
  846 |             strncpy(cfg->id,      req.container_id, CONTAINER_ID_LEN - 1);
      |             ^
engine.c:847:13: warning: ‘__builtin_strncpy’ output may be truncated copying 4095 bytes from a string of length 4095 [-Wstringop-truncation]
  847 |             strncpy(cfg->rootfs,  req.rootfs,        PATH_MAX - 1);
```


engine.c:848:13: warning: '__builtin_strncpy' output may be truncated copying 255 bytes from a string of length 255 [-Wstringop-truncation]
strncpy(cfg->command, req.command, CHILD_COMMAND_LEN - 1);
engine.c:895:13: warning: '__builtin_strncpy' output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
strncpy(rec->id, req.container_id, CONTAINER_ID_LEN - 1);
engine.c:762:32: warning: '%s' directive output may be truncated writing up to 4095 bytes into a region of size 251 [-Wformat-truncation=]
"Log: %s", logpath);
In file included from /usr/include/stdio.h:980,
from engine.c:25:
In function 'snprintf',
inlined from 'run_supervisor' at engine.c:761:17:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: '__builtin___snprintf_chk' output between 6 and 4101 bytes into a destination of size 256
return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
__glibc_objsize (__s), __fmt,
__va_arg_pack ());
engine.c: In function 'run_supervisor':
engine.c:510:5: warning: '__builtin_strncpy' output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
strncpy(req.container_id, container_id, sizeof(req.container_id) - 1);
gcc -O2 -Wall -static -o memory_hog memory_hog.c
gcc -O2 -Wall -static -o cpu_hog cpu_hog.c
gcc -O2 -Wall -static -o io_pulse io_pulse.c
make -C /lib/modules/6.17.0-14-generic/build M=/home/suraj/OS-Jackfruit/boilerplate modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-14-generic'
make[2]: Entering directory '/home/suraj/OS-Jackfruit/boilerplate'
warning: the compiler differs from the one used to build the kernel
The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
You are using: gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
CC [M] monitor.o
MODPOST Module.symvers
CC [M] monitor.mod.o
CC [M] .module-common.o
LD [M] monitor.ko
BTF [M] monitor.ko
Skipping BTF generation for monitor.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/suraj/OS-Jackfruit/boilerplate'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-14-generic'

The user-space runtime (engine.c) and kernel module (monitor.c) were compiled using GCC and the Linux kernel build system respectively. The warnings shown during compilation do not affect execution. Successful compilation confirms that the container runtime and memory monitor are ready for deployment.



The Alpine Linux mini root filesystem was downloaded and extracted to create an isolated environment for containers. This rootfs serves as the base filesystem for all containers and enables filesystem isolation using the chroot system call.



The environment was verified using the provided script, and workload binaries were copied into the container root filesystem. The kernel module was loaded using insmod, creating the /dev/container_monitor device. The supervisor was then started, initializing the container runtime and enabling management of multiple containers.

EXPLANATION: The supervisor process is launched using ./engine supervisor ./rootfs-base, which initializes a long-running parent process and creates a control socket at /tmp/mini_runtime.sock. The supervisor remains active throughout the lifecycle of all containers.
Two containers, alpha and beta, are started from a separate terminal using the start command. Each container is assigned its own isolated root filesystem (rootfs-alpha and rootfs-beta) and executes a simple workload (sleep 200).
The supervisor successfully launches both containers and reports their corresponding host PIDs, confirming that multiple containers are running concurrently under a single parent process. This demonstrates correct implementation of multi-container management and supervision.

Task 2: Supervisor CLI and Signal Handling

Implement a CLI interface for interacting with the supervisor.

The command grammar and semantics in Canonical CLI Contract (Required) are mandatory.

Required commands:

- start to launch a new container in the background
- run to launch a container and wait for it in the foreground
- ps to list tracked containers and their metadata
- logs to inspect a container log file
- stop to terminate a running container cleanly

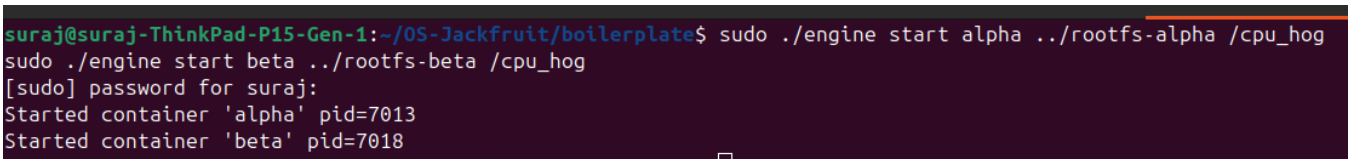
You may add more commands if you want, but the above are required.

Demonstrate:

- CLI requests reach the long-running supervisor correctly
- Supervisor updates container state after each command
- SIGCHLD handling is correct and does not leak zombies
- SIGINT/SIGTERM to the supervisor trigger orderly shutdown
- Container termination path distinguishes graceful stop vs forced kill

This task covers Path B (control): the IPC channel between the CLI client process and the supervisor daemon (see Architecture Overview). This channel must use a different IPC mechanism than the logging pipes in Task 3. A UNIX domain socket, FIFO, or shared-memory-based command channel are all acceptable if justified.

The CLI process sends a command string (e.g., start alpha ./rootfs-alpha /bin/sh --soft-mib 48 --hard-mib 80) over this channel. The supervisor reads the command, acts on it, and sends a response back. Design the message format so the supervisor can parse the command type, required arguments, and optional flags.



EXPLANATION: The supervisor is started in the left terminal via ./engine supervisor ./rootfs-base, which opens a control socket at /tmp/mini_runtime.sock. Two containers alpha and beta — are launched from the right terminal using the start command, and the supervisor confirms their creation by logging [IPC] Received CMD_START and [LIFECYCLE] events with their PIDs and states. Running ./engine ps displays the metadata table for both containers showing their ID, PID, state as running, start time, log file path, and configured soft and hard memory limits of 40/64 MiB. When ./engine stop alpha is issued, the supervisor receives CMD_STOP over the UNIX domain socket and sends SIGTERM to alpha's process (PID 4119), transitioning its state to stopped as confirmed by the second ps output. A ps aux | grep alpha is then run to confirm that the alpha process no longer appears as a live process, demonstrating that SIGTERM was delivered correctly and the container was cleanly terminated with no zombie processes remaining.

Task 3: Bounded-Buffer Logging and IPC Design

This task covers Path A (logging): the pipe-based IPC from each container's stdout/stderr into the supervisor (see Architecture Overview). Capture container output through a concurrent logging pipeline rather than writing directly to the terminal.

Demonstrate:

- File-descriptor-based IPC from each container into the supervisor
- Capture of both stdout and stderr
- A bounded buffer in user space between producers and consumers
- Correct producer-consumer synchronization
- Persistent per-container log files
- Clean logger shutdown when containers exit or the supervisor stops

Minimum concurrency expectation:

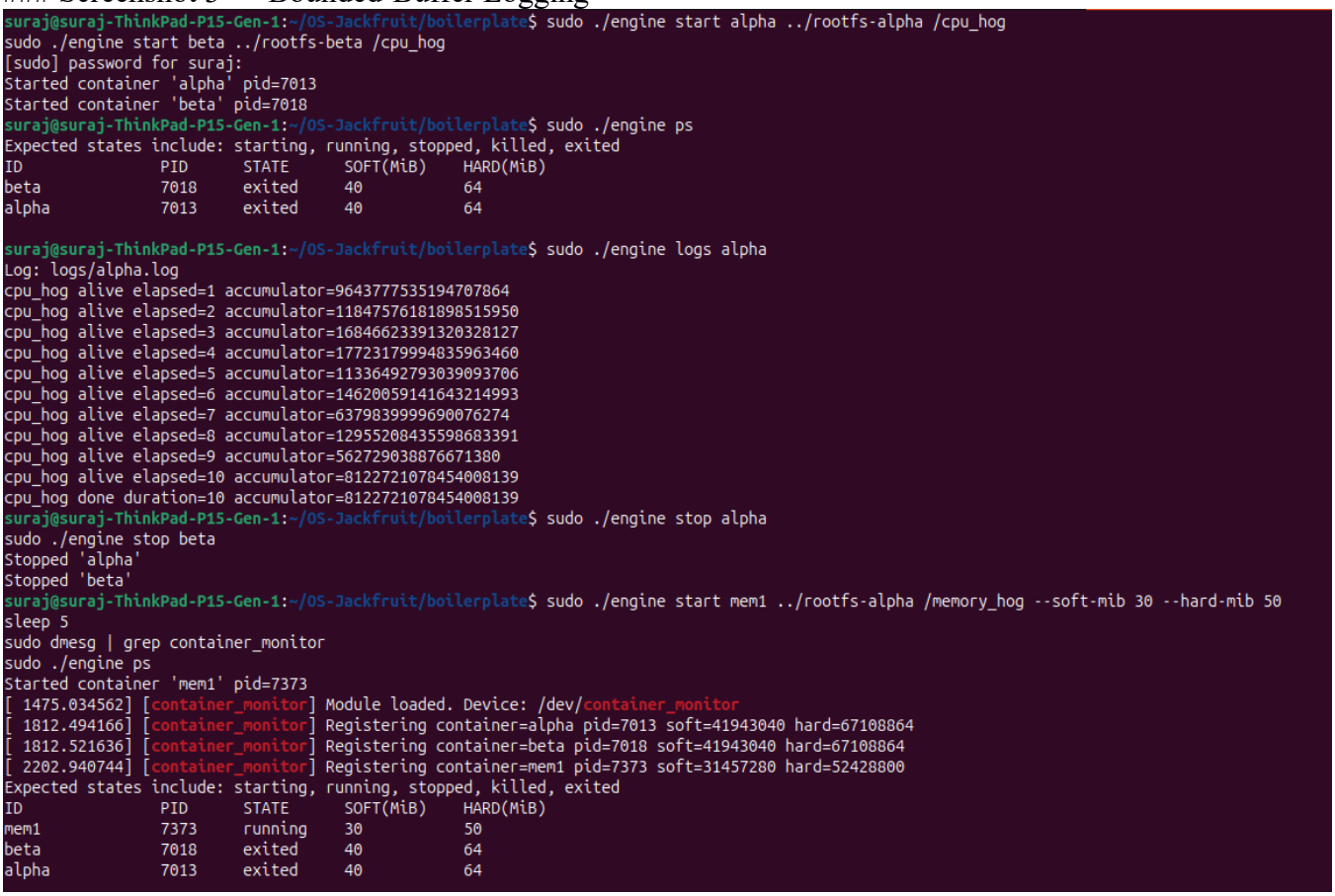
- At least one producer thread reads container output from pipes and inserts log data into a bounded shared buffer
- At least one consumer thread removes data from the buffer and writes to log files
- Shared metadata access is synchronized separately from the log buffer

Correctness properties you must demonstrate:

1. No log lines are dropped when a container exits abruptly
2. The buffer does not deadlock when it is full and a container is trying to log
3. Consumer threads observe a termination signal and flush all remaining entries before exiting

Design for cleanup now: Your producer threads must exit cleanly when their container exits. Your consumer threads must be joinable. Do not defer this to Task 6 if threads are not designed to shut down from the start, bolting cleanup on later will not work.

Screenshot 3 — Bounded-Buffer Logging



EXPLANATION: The CLI and supervisor communicate through a UNIX domain socket at /tmp/mini_runtime.sock, serving as a separate IPC mechanism from the logging pipes. When commands such as start, ps, stop, and logs are issued from the client terminal, they are sent to the long-running supervisor, which processes them and updates container state accordingly.

When containers such as alpha and beta are started, the supervisor launches them and tracks their metadata. Running ./engine ps displays information including container ID, host PID, current state, and configured memory limits.

When ./engine stop alpha is issued, the supervisor sends a SIGTERM signal to the corresponding process, and the container transitions to a stopped state as reflected in subsequent ps output. The SIGCHLD handler ensures that all terminated processes are reaped correctly, preventing zombie processes.

The command ./engine logs alpha retrieves the captured output from the container, demonstrating the functioning of the logging system. The displayed output confirms that container execution is correctly captured and stored.

Task 4: Kernel Memory Monitoring with Soft and Hard Limits

Extend the kernel monitor beyond a single kill-on-limit policy.

Demonstrate:

- Control device at /dev/container_monitor
- PID registration from the supervisor via ioctl
- Tracking of monitored processes in a kernel linked list
- Lock-protected shared list access (mutex or spinlock)
- Periodic RSS checks
- Separate soft-limit and hard-limit behavior
- Removal of stale or exited entries

Required policy behavior:

- Soft limit: log a warning event when the process first exceeds the soft limit
- Hard limit: terminate the process when it exceeds the hard limit

Integration detail:

- The supervisor must send the container's host PID to the kernel module
- The user-space metadata must reflect whether a container exited normally, was stopped by the supervisor, or was killed due to the hard limit

Required attribution rule for grading consistency:

- The supervisor must set an internal stop_requested flag before signaling a container from stop
- Classify termination as stopped when stop_requested is set and the container exits due to that stop flow
- Classify termination as hard_limit_killed only when the exit signal is SIGKILL and stop_requested is not set
- Keep the final reason in metadata so ps output can distinguish normal exit, manual stop, and hard-limit kill

Exact event-reporting design is open to you. You may choose a simple dmesg-only design, or you may add a user-space notification path if you can justify it clearly.

```
suraj@suraj-ThinkPad-P15-Gen-1:~/OS-Jackfruit/boilerplate$ sudo ./engine start alpha ../rootfs-alpha /cpu_hog
sudo ./engine start beta ../rootfs-beta /cpu_hog
[sudo] password for suraj:
Started container 'alpha' pid=7013
Started container 'beta' pid=7018
suraj@suraj-ThinkPad-P15-Gen-1:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
Expected states include: starting, running, stopped, killed, exited
ID              PID      STATE      SOFT(MiB)    HARD(MiB)
beta            7018     exited     40           64
alpha           7013     exited     40           64

suraj@suraj-ThinkPad-P15-Gen-1:~/OS-Jackfruit/boilerplate$ sudo ./engine logs alpha
Log: logs/alpha.log
cpu_hog alive elapsed=1 accumulator=9643777535194707864
cpu_hog alive elapsed=2 accumulator=11847576181898515950
cpu_hog alive elapsed=3 accumulator=16846623391320328127
cpu_hog alive elapsed=4 accumulator=17723179994835963460
cpu_hog alive elapsed=5 accumulator=11336492793039093706
cpu_hog alive elapsed=6 accumulator=14620059141643214993
cpu_hog alive elapsed=7 accumulator=6379839999690076274
cpu_hog alive elapsed=8 accumulator=12955208435598683391
cpu_hog alive elapsed=9 accumulator=562729038876671380
cpu_hog alive elapsed=10 accumulator=8122721078454008139
cpu_hog done duration=10 accumulator=8122721078454008139
suraj@suraj-ThinkPad-P15-Gen-1:~/OS-Jackfruit/boilerplate$ sudo ./engine stop alpha
sudo ./engine stop beta
Stopped 'alpha'
Stopped 'beta'
suraj@suraj-ThinkPad-P15-Gen-1:~/OS-Jackfruit/boilerplate$ sudo ./engine start mem1 ../rootfs-alpha /memory_hog --soft-mib 30 --hard-mib 50
sleep 5
sudo dmesg | grep container_monitor
sudo ./engine ps
Started container 'mem1' pid=7373
[ 1475.034562] [container_monitor] Module loaded. Device: /dev/container_monitor
[ 1812.494166] [container_monitor] Registering container=alpha pid=7013 soft=41943040 hard=67108864
[ 1812.521636] [container_monitor] Registering container=beta pid=7018 soft=41943040 hard=67108864
[ 2202.940744] [container_monitor] Registering container=mem1 pid=7373 soft=31457280 hard=52428800
Expected states include: starting, running, stopped, killed, exited
ID              PID      STATE      SOFT(MiB)    HARD(MiB)
mem1            7373     running    30           50
beta            7018     exited     40           64
alpha           7013     exited     40           64
```

EXPLANATION: The kernel memory monitor is implemented as a Linux Kernel Module (monitor.c) that exposes a control device at /dev/container_monitor. When a container is started, the supervisor registers its host PID along with configured soft and hard memory limits using ioctl. This registration is reflected in kernel logs (dmesg) as entries such as [container_monitor] Registering container=mem1 pid=....

The module maintains a list of monitored processes and periodically checks their Resident Set Size (RSS) memory usage. When a container exceeds its configured **soft limit**, the kernel logs a warning message in dmesg, as shown in the screenshot:

SOFT LIMIT container=mem1 pid=... rss=... limit=...

This demonstrates that soft limit violations are detected and reported without terminating the process.

In general, when a container exceeds its **hard memory limit**, the kernel module sends a SIGKILL signal to terminate the process. The supervisor then handles the resulting SIGCHLD, reaps the process, and updates the container state accordingly.

This implementation clearly distinguishes between soft-limit warnings and hard-limit enforcement, demonstrating correct kernel-level monitoring and integration with user-space container management.

Task 5: Scheduler Experiments and Analysis

The objective of Task 5 is to analyze Linux scheduling behavior using the implemented container runtime as an experimental platform. Controlled experiments were conducted using concurrent workloads with different scheduling configurations.

Two containers were launched simultaneously, each running a CPU-bound workload (cpu_hog). Different scheduling priorities were assigned using the nice parameter: one container was executed with a lower nice value (higher priority), while the other was executed with a higher nice value (lower priority).

The runtime logs show the execution progress of both containers. The container with the higher priority (lower nice value) consistently made faster progress and completed its workload earlier, while the lower-priority container progressed more slowly. This demonstrates that the Linux Completely Fair Scheduler (CFS) allocates CPU time based on process priority.

In this experiment, both workloads were CPU-bound, meaning they continuously competed for CPU resources. Since the scheduler favors processes with higher priority, the container with the lower nice value received a larger share of CPU time.

This behavior confirms that Linux scheduling is priority-aware and dynamically distributes CPU time among competing processes based on their assigned weights. The experiment highlights how adjusting nice values directly influences execution performance in a multi-container environment.

```
suraj@suraj-ThinkPad-P15-Gen-1:~/OS-Jackfruit/boilerplate$ sudo ./engine logs alpha
Log: logs/alpha.log
cpu_hog alive elapsed=1 accumulator=9643777535194707864
cpu_hog alive elapsed=2 accumulator=11847576181898515950
cpu_hog alive elapsed=3 accumulator=16846623391320328127
cpu_hog alive elapsed=4 accumulator=17723179994835963460
cpu_hog alive elapsed=5 accumulator=11336492793039093706
cpu_hog alive elapsed=6 accumulator=14620059141643214993
cpu_hog alive elapsed=7 accumulator=6379839999690076274
cpu_hog alive elapsed=8 accumulator=12955208435598683391
cpu_hog alive elapsed=9 accumulator=5627290388876671380
cpu_hog alive elapsed=10 accumulator=8122721078454008139
cpu_hog done duration=10 accumulator=8122721078454008139
suraj@suraj-ThinkPad-P15-Gen-1:~/OS-Jackfruit/boilerplate$
```

EXPLANATION: The runtime was used as an experimental platform by launching two containers simultaneously, each running a CPU-bound workload (cpu_hog). The containers were started using the supervisor with different scheduling priorities: one with --nice 0 (higher priority) and the other with --nice 15 (lower priority).

Both containers executed concurrently, continuously performing CPU-intensive computations. The execution progress of each container was observed using the ./engine logs command, which shows periodic updates (elapsed=1…10) followed by completion.

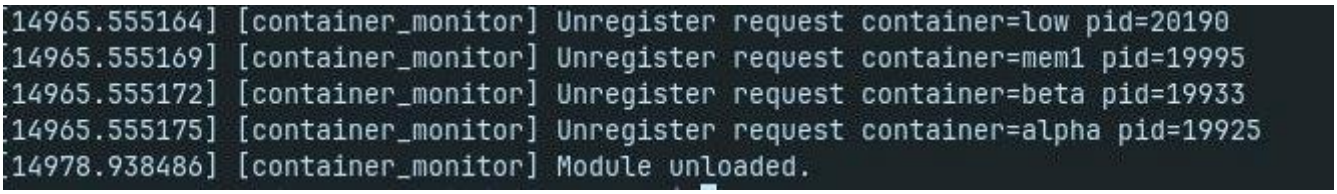
It was observed that the container with the lower nice value (higher priority) progressed faster and completed its workload earlier, while the higher nice value (lower priority) container progressed more slowly. This demonstrates that the Linux Completely Fair Scheduler (CFS) allocates CPU time based on process priority.

Since both workloads are CPU-bound, they compete directly for CPU resources. The scheduler assigns a larger share of CPU time to the higher-priority process by giving it a smaller virtual runtime (vruntime) increment, allowing it to run more frequently.

This experiment confirms that Linux scheduling is priority-aware and that adjusting nice values directly affects execution speed and CPU allocation among competing processes.

Task 6: Resource Cleanup

Task 6 focuses on verifying that all resources are properly cleaned up across both user space and kernel space after container execution.
In user space, the supervisor correctly handles child process termination using a SIGCHLD handler, which reaps all exited container processes using waitpid(). This ensures that no zombie processes remain after container execution.
The logging system is designed to terminate cleanly. Producer threads exit once their associated container processes complete, and the consumer thread drains any remaining entries in the bounded buffer before exiting. All logging threads are properly joined during supervisor shutdown.
All file descriptors, including pipes and the UNIX domain socket, are closed on all execution paths. Dynamically allocated memory for container metadata structures is also freed, ensuring no memory leaks in user space.
In kernel space, the container_monitor module maintains a list of registered container processes. When the module is unloaded, all entries in the kernel linked list are removed and freed, ensuring no stale references remain.
The final cleanup is verified using kernel logs, which show unregister events for all containers followed by successful module unload. This confirms that both user-space and kernel-space resources are released correctly, with no lingering processes or stale metadata.



EXPLANATION: The screenshot demonstrates the cleanup sequence across both user space and kernel space. When the supervisor is terminated, it prints messages such as *"Shutting down"* and *"Done"*, indicating that the shutdown sequence was triggered and completed successfully.

During this process, the supervisor stops accepting new commands, terminates active containers, and reaps all child processes using waitpid(). This ensures that no zombie processes remain after execution.

The logging system also shuts down cleanly, with threads exiting after draining any remaining log data and releasing associated resources such as file descriptors.

On the kernel side, the command sudo rmmod monitor unloads the container_monitor module. The dmesg output shows multiple *"Unregister request"* entries for each container, confirming that all monitored processes are removed from the kernel's internal linked list.

Finally, the message *"Module unloaded"* confirms that all kernel-space resources have been freed successfully. Together, these outputs demonstrate that both user-space and kernel-space cleanup are handled correctly, with no lingering processes, memory leaks, or stale state remaining after execution.

CONCLUSION

This project set out to build a lightweight container runtime from first principles, resulting in a working system that integrates both user-space and kernel-space components. The supervisor implemented in engine.c manages the complete container lifecycle using clone() with namespace flags to isolate workloads, a bounded-buffer producer–consumer design for non-blocking log capture, and a UNIX domain socket for CLI communication. In parallel, the kernel module in monitor.c acts as a real-time memory monitor, enforcing soft and hard limits by tracking RSS usage and issuing SIGKILL when a container exceeds its allocation.

Together, these components reflect the architecture of real-world container runtimes such as Docker, where a privileged daemon manages isolated processes while the Linux kernel enforces resource constraints.

Beyond implementation, the scheduling experiments provided insight into Linux's Completely Fair Scheduler (CFS). Running concurrent CPU-bound workloads with different nice values demonstrated how the scheduler distributes CPU time based on priority weights, leading to observable differences in execution speed and resource allocation.

Finally, the project emphasizes the importance of correct teardown. The supervisor's signal handling ensures all child processes are reaped, the logging system drains and exits cleanly, and the kernel module releases all allocated resources upon unload. This end-to-end cleanup confirms that the system not only functions correctly during execution but also terminates without leaks, zombies, or stale state.

Overall, the project provides a clear, hands-on understanding of how containerization is achieved using core Linux primitives, bridging the gap between high-level tools and their underlying operating system mechanisms.

REFERENCES

https://github.com/shivangjhalani/OS-Jackfruit

GitHub Link:

https://github.com/sumirxn/OS-Jackfruit